FABRICARE

Laundry Management System

PROJECT DOCUMENTATION

Team	Software Development Team Documentation
Project Name	FabriCare (Laundrix Ecosystem)
Department	Software Development Team
Prepared By	Pratiksha Harode, Priyanka Kushwah
Date	10.11.2026

A connected workflow for customer booking, pickup, laundry processing and final delivery.

Technology direction: React 18 + Vite · Django 5.1 REST Framework · Firebase Realtime Database & SQLite / PostgreSQL

1. Project Overview

FabriCare is a modern laundry and dry cleaning management platform connecting four user roles (Customer, Administrator, Laundry Staff, and Delivery Partner) through a unified real-time order lifecycle. This document defines the actual portal structure, technical architecture, and end-to-end operational workflow.

Project objectives

- Allow customers to choose laundry services, select items with custom quantities, and place orders with verified pickup address and scheduled time slot.
- Allow admin to review new orders in real time, accept or reject them, assign pickup executives, and monitor hub throughput.
- Allow delivery staff to manage assigned pickup tasks, verify customer collection via secure 4-digit email OTP, and confirm handover at the laundry hub.
- Allow laundry staff to verify garment counts, track live processing stages (Washing, Drying, Ironing), perform quality checks, and pack orders.
- Assign completed orders for final doorstep delivery, verify delivery OTP, record payment collection, and complete order lifecycle.
- Maintain one unified order ID with real-time status synchronization across all four portals via Firebase Realtime Database and Django REST APIs.

Technology direction

Layer	Technology	Responsibility
Frontend	React 18 · Vite 6 · Tailwind CSS · Lucide React	Role-based portal interfaces, dynamic multi-step booking, real-time order tracking, camera image uploads and responsive styling.
Backend	Django 5.1 · Django REST Framework · SimpleJWT	Business logic, role permissions, state machine transitions, REST API endpoints, user authentication and database models.
Database & Realtime	Firebase Realtime Database & SQLite / PostgreSQL	Instant two-way live sync for active orders, real-time status transitions, OTP security secrets, and persistent relational data storage.

This document describes the implemented structure and verified workflows of the FabriCare Laundry Management Platform.

2. Portal Structure and Responsibilities

Each portal contains the pages and views required by its specific user role. The folder hierarchies below show the actual source files and organizational structure across the frontend codebase.

2.1 Customer Portal

Folder: frontend/src/customer/

```
customer/
├── CustomerDashboard.jsx      # Customer overview, active orders, quick actions
├── BookingConfirmation.jsx    # Booking receipt and order confirmation view
├── MyOrders.jsx               # Active, ongoing, and past order history
├── TrackOrder.jsx             # Live visual timeline and order tracking stepper
├── SavedAddresses.jsx         # Doorstep pickup & delivery address book
├── PaymentsInvoices.jsx       # Order billing, invoice downloads, payment records
├── HelpSupport.jsx            # Support ticket creation and FAQs
├── CustomerReports.jsx        # Customer complaints and issue reporting
├── Notifications.jsx          # Order lifecycle updates and account alerts
└── Profile.jsx                # Personal details, phone, and security settings
```

Features

- Select laundry services (Wash & Fold, Dry Cleaning, Steam Press, Premium Fabric Care) with customized garment item quantities.
- Choose doorstep pickup address, add landmark details, and select preferred pickup date and time slots.
- Track order progress in real time through an interactive visual stepper synced with Firebase Realtime Database.
- View detailed itemized bills, pricing breakdown (service total, pickup fee, discounts, taxes), and download PDF invoices.
- Manage saved addresses, profile settings, notifications, support tickets, and complaint resolutions.

2.2 Admin Portal

Folder: frontend/src/pages/admin/

```
admin/
├── AdminDashboardPage.jsx     # Live KPIs, order pipeline, revenue stats, hub load
├── AdminOrdersPage.jsx        # Comprehensive orders management and status controls
├── AdminPickupDeliveryPage.jsx# Pickup and delivery task dispatching and assignment
├── AdminCustomersPage.jsx     # Registered customers management and order histories
├── AdminUsersPage.jsx         # Laundry staff & delivery agents role management
├── AdminServicesPage.jsx      # Laundry services catalog, garment items & pricing
├── AdminDeliveryReportsPage.jsx# Delivery performance metrics, completion rates
├── AdminComplaintReportsPage.jsx# Customer complaints investigation and rescheduling
├── AdminNotificationsPage.jsx # Broadcast alerts and system notifications manager
└── AdminSettingsPage.jsx      # Platform configuration, operational hours, hubs
```

Features

- Review new customer orders in real time with quick Accept, Reject, or Reschedule capabilities.
- Assign available delivery executives for doorstep customer pickups and final order deliveries.
- Manage services catalog, itemized rates, express turnaround fees, and tax calculations.
- Monitor operational health: active orders, hub workload, pending pickups, and delayed delivery alerts.
- Manage registered users, review staff identity documents (Aadhaar, Driving License, PAN), and audit activity.

2.3 Laundry Staff Portal

Folder: frontend/src/laundry-staff/

```
laundry-staff/
├── LaundryPortal.jsx          # Hub staff workspace, stage tabs and modal handlers
├── Dashboard.tsx              # Processing queues overview, urgent orders count
├── Orders.tsx                 # Processing queues (Washing, Drying, Ironing, Packing)
├── OrderDetails.tsx           # Garment breakdown, special wash instructions, notes
├── OrderHistory.tsx           # Completed processing batch history and archive
├── Notifications.tsx          # Real-time alerts for incoming driver handovers
└── taskConfig.js              # Processing stage definitions, colors, and workflows
```

Features

- Confirm receipt of clothes from delivery executives at the hub and verify physical item counts against customer order.
- Manage stage-by-stage garment processing pipeline: Intake → Washing → Drying → Steam Ironing.
- Record Quality Check (QC) inspection results: pass garments to packing or route defective items for rework.
- Pack completed garments, generate packaging verification tags, and mark orders as Ready for Delivery.
- Track processing stage timestamps and staff accountability for every garment batch.

2.4 Delivery Portal

Folder: frontend/src/delivery/

```
delivery/
├── DeliveryPortal.jsx         # Executive task workspace, OTP verification & modals
├── MyPickupTasks.tsx          # Assigned doorstep pickup tasks list with live filters
├── PickupDetails.tsx          # Customer address, phone, item count, pickup notes
├── PickupConfirmation.tsx     # 4-Digit customer OTP validation, bags count & weight
├── LaundryDropOff.tsx         # Handover confirmation at laundry hub
├── MyDeliveryTasks.tsx        # Assigned doorstep delivery tasks list
├── DeliveryDetails.tsx        # Customer location, payment status, deliver notes
├── DeliveryConfirmation.tsx   # Final 4-digit delivery OTP verification & proof
├── FailedDeliveries.tsx       # Failed attempt logger with rescheduling reasons
└── TaskHistory.tsx            # Completed delivery runs, earnings, and date filters
```

Features

- View assigned pickup and delivery tasks with customer details, phone dialer, navigation address, and slot time.
- Secure Doorstep Pickup: Verifies customer identity via real 4-digit OTP dispatched to customer email (EmailJS API).
- Record collected laundry bags count, approximate weight (kg), and condition notes at customer doorstep.
- Confirm safe handover and drop-off of collected garments to laundry hub staff.
- Secure Final Delivery: Verifies customer delivery OTP, records recipient name, collects cash/UPI, and saves delivery proof.

3. End-to-End Order Workflow

Pickup and final delivery are separate operational tasks, but both remain linked to the same master order record and order ID, synchronized in real time across all four portals.

Step	Responsible portal	Action	Example status
1	Customer	Customer selects services, items, specifies pickup address and time slot, and confirms order.	PENDING / Order Placed
2	Admin	Admin reviews incoming order details and accepts or rejects the order.	ACCEPTED / REJECTED
3	Admin → Delivery	Admin dispatches and assigns an active delivery partner for customer doorstep pickup.	PICKUP_ASSIGNED
4	Delivery	Partner reaches doorstep, verifies real 4-digit customer OTP sent to email, records bags and weight.	PICKED_UP
5	Delivery → Laundry Staff	Partner drops garments at laundry hub; staff verifies physical items and confirms receipt.	AT_HUB / Received at Hub
6	Laundry Staff	Staff advances order through washing, drying, and steam ironing stages with live progress.	IN_PROCESSING (Washing/Drying/Ironing)
7	Laundry Staff	Staff conducts quality check inspection, bags items, and marks order ready for dispatch.	READY_FOR_DELIVERY / Packed
8	Admin / System → Delivery	Admin assigns a delivery executive to transport packed clothes from hub to customer.	OUT_FOR_DELIVERY
9	Delivery	Partner delivers clean clothes at doorstep, verifies 4-digit delivery OTP, and collects payment.	DELIVERED / Completed

Exception handling

- Rejected orders do not proceed to pickup; customer is notified instantly with rejection reason.
- Failed pickup or delivery attempts (e.g. customer unavailable, wrong address) are recorded with failure reasons and automatically queued for Admin rescheduling.
- Garments failing quality check inspection are returned to Rework stage with issue comments rather than being packed.
- Every status transition automatically records timestamp, responsible user ID, and updates Firebase Realtime Database for zero-latency client sync.

4. Technical Architecture

The FabriCare architecture combines a reactive React 18 frontend, Django 5.1 REST Framework for backend business logic, and Firebase Realtime Database for live cross-portal event synchronization.

Layer	Technology	Responsibility
Frontend	React 18 · Vite 6 · Tailwind CSS · Context API	Single Page Application (SPA), role-based routing, real-time tracking views, modal dialogs, and responsive layout.
Backend	Django 5.1 · Django REST Framework · SimpleJWT	Authoritative business rules, permissions validation, user management, order state machine, and RESTful APIs.
Database & Realtime	Firebase Realtime Database & SQLite / PostgreSQL	High-speed live order synchronization, active delivery task status events, security secrets (OTPs), and relational persistence.
External Services	EmailJS REST API · Firebase Storage	Automated delivery of real 4-digit customer verification OTPs directly to inbox, and cloud storage for delivery/agent photos.

Database connection & Real-Time Sync

Django connects to the relational database (SQLite in development, PostgreSQL in production) through backend environment variables. Simultaneously, Firebase Realtime Database synchronizes state changes across active browser clients instantaneously without polling.

Security principles

- Store sensitive credentials (Firebase Admin keys, JWT secret keys, EmailJS tokens) strictly in backend and frontend .env files.
- Enforce strict Role-Based Access Control (RBAC) on the backend using Django permissions and custom JWT claims.
- Secure doorstep pickup and delivery handovers using real 4-digit OTPs generated securely and dispatched directly to customer email.
- Never expose privileged administrative database keys or service credentials in client-side bundles.

Actual backend and frontend codebase structure

```
FabriCare/
├── backend/
│   ├── manage.py
│   ├── db.sqlite3
│   ├── .env
│   ├── config/
│   │   ├── settings/ (base.py, dev.py, prod.py)
│   │   ├── urls.py
│   │   └── wsgi.py
│   └── apps/
│       ├── core/ (models, serializers, views, admin)
│       └── users/ (models, authentication, serializers, views)
└── frontend/
    ├── package.json
    ├── vite.config.js
    ├── .env
    └── src/
        ├── api/ (apiClient.js)
        ├── config/ (firebase.js)
        ├── context/ (AuthContext, AdminContext, BookingContext)
        ├── customer/ (Dashboard, MyOrders, TrackOrder, Booking)
        ├── pages/admin/ (Dashboard, Orders, Dispatch, Users, Services)
        ├── laundry-staff/ (LaundryPortal, Orders, taskConfig)
        ├── delivery/ (DeliveryPortal, MyPickupTasks, TaskHistory)
        └── services/ (otpEmailService, firestoreService, authService)
```

5. Core Data Entities

Entity	Purpose in Project Codebase (apps.core & apps.users)
User & Role	Custom user model supporting 4 distinct roles: CUSTOMER, ADMIN, LAUNDRY_STAFF, and DELIVERY_STAFF.
CustomerAddress	Doorstep pickup & delivery locations with flat/house, area/colony, city, and pincode.
ServiceCategory	Top-level laundry categories: Wash & Fold, Dry Cleaning, Steam Press, Premium Care.
Service	Specific laundry items and garment types with standard prices, unit metrics, and turnaround times.
Order & OrderItem	Master order record linking customer, selected garments, quantities, total amount, and status.
PickupRecord	Driver pickup confirmation capturing collected bag count, weight (kg), and verified customer OTP.
LaundryHandoverRecord	Hub intake record documenting garment receipt verification between delivery driver and hub staff.
LaundryStageLog	Real-time timestamp log for Washing, Drying, and Steam Ironing processing stages.
DeliveryRecord	Final delivery log recording recipient name, doorstep delivery OTP verification, and payment proof.
Invoice	Financial record containing subtotal, tax amount, delivery charges, discounts, and payment method.
Notification & SupportTicket	In-app notifications and customer support / complaint tickets with status resolution tracking.

6. API and Tracking Approach

The following are the active versioned REST API endpoints implemented in backend/config/urls.py.

Area	API Endpoint	Operational Purpose
Authentication	/api/v1/auth/login/ /api/v1/auth/register/ /api/v1/auth/me/	User authentication, JWT token issuance, session refresh, and profile retrieval.
Services Catalog	/api/v1/services/ /api/v1/services/categories/	Retrieve active laundry categories, garment items, base prices, and express rates.
Orders	/api/v1/orders/ /api/v1/customer/orders/	Create orders, list authorized customer orders, and retrieve order details.
Admin Dispatch	/api/v1/admin/orders/{id}/accept/ /api/v1/admin/orders/{id}/assign-staff/	Admin order review, acceptance/rejection, and delivery agent assignment.
Delivery Tasks	/api/v1/delivery/tasks/ /api/v1/delivery/tasks/{id}/confirm-pickup/ /api/v1/delivery/tasks/{id}/confirm-delivery/	Fetch driver task queue, verify customer pickup OTP, log drop-off, and confirm final delivery.
Laundry Processing	/api/v1/laundry/orders/ /api/v1/laundry/orders/{id}/advance-stage/ /api/v1/laundry/orders/{id}/quality-check/	Hub intake confirmation, advance washing/drying/ironing stages, and record QC check results.
Tracking & Alerts	/api/v1/orders/{id}/history/ /api/v1/notifications/ /api/v1/support/	Fetch chronological order transition history, user notifications, and support tickets.

Tracking rules

- Backend validates every status transition against an authoritative state machine before writing to the database.
- Valid state changes update the relational database and simultaneously broadcast to Firebase Realtime Database.
- Customer Track Order page listens directly to Firebase nodes for instant, real-time UI updates without polling.
- Failed quality checks automatically route orders to Rework status with mandatory staff inspection notes.
- Doorstep pickup and delivery events strictly require 4-digit OTP verification matching the database record.

7. Implementation Plan

Phase	Focus	Expected outcome
1 — Foundation & Auth	Django 5.1 project setup, Custom User model, JWT authentication, Firebase SDK integration, React Vite setup.	Secure base authentication, RBAC authorization, and multi-portal routing.
2 — Catalog & Booking	Services categories, garment items, multi-step booking wizard, address book, and price calculations.	Customers can select items, configure addresses, and submit orders.
3 — Admin & Dispatch	Live operational dashboard, pending orders queue, Accept/Reject controls, and automated driver dispatch.	Admins can review orders, assign delivery staff, and monitor hub metrics.
4 — Doorstep Pickup & OTP	Delivery tasks feed, EmailJS real 4-digit OTP dispatch, bag count logging, approximate weight and hub drop-off.	Secure doorstep pickup with real customer OTP verification and hub handover.
5 — Hub Processing & QC	Hub intake verification, stage advancing (Washing, Drying, Ironing), QC pass/rework routing, and packing.	Full transparency into laundry cleaning stages and quality verification.
6 — Delivery & Verification	Out for delivery dispatch, delivery OTP validation, cash/UPI collection, and multi-portal live sync.	Completed end-to-end order lifecycle verified across all four portals.

End-to-end verification checklist

- A customer can place an order and see it immediately in Customer Dashboard and My Orders.
- The admin reviews new orders; rejected orders are canceled, and accepted orders can be assigned for pickup.
- Only the assigned delivery partner sees the task, navigates to customer, and enters the real 4-digit OTP sent to customer email.
- Pickup confirmation records bag count, approximate weight (kg), and confirms safe handover at the laundry hub.
- Laundry staff verifies intake, advances stages (Washing, Drying, Ironing), and passes quality check inspection.
- Completed garments are packed, tagged, and marked Ready for Delivery.
- Delivery partner delivers clean clothes at customer doorstep and verifies the delivery OTP.
- The customer visualizer reflects all status updates in real time via Firebase Realtime Database without page refresh.
- Role permissions, token authentication, and API endpoints are strictly tested and verified.

Final note

This document defines the verified system architecture, real codebase components, and recommended workflows for FabriCare. Before adding new features or migrations, verify existing endpoints and database models in apps.core and apps.users to preserve data integrity.

Prepared by Pratiksha Harode, Priyanka Kushwah · Software Development Team · 10.11.2026